

Escuela Colombiana de Ingeniería Julio Garavito

Programa de Ingeniería de Sistemas

Guía de Laboratorio

Apache Kafka y Arquitecturas Orientadas por Eventos

Asignatura	Duración	Modalidad	Nivel
Arquitecturas de Software (ARSW)	2 a 3 horas	2 estudiantes	Intermedio

Introducción

Las arquitecturas modernas requieren mecanismos de comunicación que permitan desacoplar servicios, procesar grandes volúmenes de eventos y responder de forma resiliente ante fallos parciales. Apache Kafka es una plataforma de event streaming que permite publicar, almacenar y consumir eventos de manera distribuida. Esta guía introduce los fundamentos arquitectónicos, técnicos y prácticos necesarios para comprender cuándo y cómo utilizar Kafka en sistemas basados en eventos.

Objetivo general

Comprender los principios de las arquitecturas orientadas por eventos y aplicar Apache Kafka en un laboratorio práctico con Docker, Kafka UI y Spring Boot.

Resultados de aprendizaje

Al finalizar la práctica, el estudiante podrá explicar los componentes principales de Kafka, diseñar eventos y topics coherentes, implementar productores y consumidores, analizar Consumer Groups, manejar errores básicos y justificar decisiones arquitectónicas asociadas a escalabilidad, disponibilidad, mantenibilidad y observabilidad.

Componente	Descripción
Enfoque	Arquitecturas orientadas por eventos y Apache Kafka.
Herramientas	Docker, Docker Compose, Kafka UI, Java21, Maven y Spring Boot.
Producto esperado	Aplicación funcional con productor y consumidores, más una propuesta arquitectónica justificada.

Capítulo 1. Evolución hacia arquitecturas orientadas por eventos

El desarrollo de software ha evolucionado para responder a necesidades cada vez más exigentes de escalabilidad, disponibilidad y mantenibilidad. Comprender esta evolución permite entender por qué Kafka aparece como una solución relevante en sistemas distribuidos modernos.

1.1 Arquitecturas tradicionales

Estilo	Características	Limitaciones
Monolítica	Toda la aplicación se despliega como una unidad.	Escalabilidad limitada, alto acoplamiento y despliegues riesgosos.
Cliente-servidor	Los clientes solicitan información a un servidor central.	Dependencia del servidor y posibles cuellos de botella.
Capas	Separación entre presentación, negocio, persistencia y datos.	Puede crecer como un monolito si no se controla el acoplamiento.

1.2 SOA y microservicios

La Arquitectura Orientada a Servicios (SOA) propuso integrar capacidades empresariales mediante servicios reutilizables. Posteriormente, los microservicios llevaron esta idea a unidades más pequeñas, autónomas y desplegables de manera independiente.

Principio clave

Un microservicio debe concentrarse en una capacidad del negocio. Esta responsabilidad clara mejora la mantenibilidad y permite escalar cada servicio de forma independiente.

1.3 El problema de la comunicación

Cuando los servicios se comunican exclusivamente mediante llamadas REST síncronas, cada servicio debe esperar la respuesta del siguiente. Esto puede crear cadenas de dependencia, aumentar la latencia y reducir la tolerancia a fallos.

Cliente -> Order Service -> Payment Service -> Inventory Service -> Notification Service -> Respuesta

1.4 Comunicación síncrona y asíncrona

Aspecto	Síncrona (REST)	Asíncrona (eventos)
Relación	El cliente espera respuesta.	El productor publica y continúa.
Acoplamiento	Mayor acoplamiento temporal.	Menor acoplamiento entre servicios.
Uso típico	Consultas y validaciones inmediatas.	Procesos posteriores, integración y notificaciones.
Riesgo	Fallas en cadena.	Consistencia eventual y mayor complejidad operativa.

1.5 Arquitectura orientada por eventos

Una arquitectura orientada por eventos es un estilo en el cual los componentes publican y consumen eventos. Un evento representa un hecho relevante que ya ocurrió en el dominio, por ejemplo: order-created, payment-approved o inventory-reserved.

Order Service -> evento order-created -> Broker de eventos -> Payment / Inventory / Notification / Analytics

1.6 Productores, consumidores y broker

Elemento	Responsabilidad
Productor	Publica eventos cuando ocurre un hecho relevante.

Consumidor	Lee eventos y reacciona según su responsabilidad.
Broker	Recibe eventos de productores y los distribuye a consumidores interesados.
Topic	Categoría lógica donde se almacenan eventos relacionados.

1.7 De la mensajería al event streaming

En una cola tradicional, un mensaje suele ser consumido por un trabajador y luego deja de estar disponible. En event streaming, los eventos permanecen durante un periodo definido, lo que permite que varios consumidores los lean, los reprocesen o los utilicen con fines de analítica y auditoría.

Actividad 1. Análisis de comunicación

Para una tienda en línea, clasifique qué procesos deberían ser síncronos, asíncronos o híbridos: consultar productos, crear pedido, validar pago, enviar notificación, actualizar analítica y registrar auditoría. Justifique brevemente su decisión.

Capítulo 2. Apache Kafka: fundamentos y arquitectura interna

Apache Kafka es una plataforma distribuida de event streaming. Permite publicar, almacenar, procesar y consumir eventos a gran escala. No debe entenderse únicamente como una cola de mensajes, sino como un log distribuido de eventos.

2.1 Idea central

Productor -> Topic en Kafka -> Consumidores independientes

El productor no necesita conocer a los consumidores. Cada consumidor puede leer los eventos a su propio ritmo y mantener su propia posición de lectura.

2.2 Componentes principales

Concepto	Definición
Broker	Servidor Kafka encargado de recibir, almacenar y entregar eventos.
Cluster	Conjunto de brokers que trabajan de forma coordinada.
Topic	Categoría lógica de eventos.
Partición	División física de un topic que permite paralelismo.
Offset	Posición de un evento dentro de una partición.
Producer	Aplicación que publica eventos.
Consumer	Aplicación que consume eventos.
Consumer Group	Grupo de consumidores que reparte el trabajo sobre particiones.
Replica	Copia de una partición en otro broker.
Leader	Réplica principal que atiende lecturas y escrituras.
Follower	Réplica secundaria que copia datos del líder.
ISR	Réplicas sincronizadas con el líder.
Retention	Tiempo o tamaño durante el cual Kafka conserva eventos.
KRaft	Modo moderno de coordinación de Kafka sin ZooKeeper.

2.3 Topics y particiones

Un topic puede dividirse en varias particiones. Kafka garantiza el orden de los eventos dentro de una misma partición, pero no garantiza un orden global entre todas las particiones del topic.

Topic orders

Partition 0: offset 0, offset 1, offset 2

Partition 1: offset 0, offset 1, offset 2

Partition 2: offset 0, offset 1, offset 2

2.4 Clave de particionamiento

La clave permite enviar eventos relacionados a la misma partición. Si se requiere conservar el orden de los eventos de un pedido, una buena clave puede ser orderId.

Dominio	Clave sugerida
Pedidos	orderId
Cuentas bancarias	accountId
Estudiantes	studentId
Vehículos	vehicleId
Productos	productId

2.5 Consumer Groups

Dentro de un mismo Consumer Group, cada partición es procesada por un solo consumidor. Si dos servicios necesitan recibir el mismo evento, deben pertenecer a grupos diferentes.

Topic orders

Grupo payment-service -> consume order-created

Grupo inventory-service -> consume order-created

Grupo analytics-service -> consume order-created

2.6 Replicación, retención y disponibilidad

En laboratorio es aceptable usar un solo broker y factor de replicación 1. En producción, un factor de replicación mayor permite tolerar fallos. La retención define durante cuánto tiempo los eventos permanecen disponibles para lectura o reprocesamiento.

Actividad 2. Decisiones de configuración

Analice una configuración con un topic orders, una partición, factor de replicación 1, mensajes sin clave y retención de 24 horas. Identifique riesgos y proponga mejoras para un ambiente productivo.

Capítulo 3. Preparación del entorno de laboratorio

El laboratorio utiliza Docker para ejecutar Kafka en modo KRaft y Kafka UI como herramienta visual de inspección. Esto evita instalaciones complejas y permite trabajar con un entorno homogéneo.

3.1 Requisitos

- Docker y Docker Compose instalados.
- Java 17 o superior.
- Maven 3.8 o superior.
- Editor de código como IntelliJ IDEA o Visual Studio Code.
- Conocimientos básicos de terminal y Spring Boot.

3.2 Docker Compose

services:

kafka:

```

image: apache/kafka:3.7.0
container_name: arsw-kafka
ports:
  - "9092:9092"
environment:
  KAFKA_NODE_ID: 1
  KAFKA_PROCESS_ROLES: broker,controller
  KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
  KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
  KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
  KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
  KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
  KAFKA_NUM_PARTITIONS: 3

```

```

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  container_name: arsw-kafka-ui
  ports:
    - "8080:8080"
  depends_on:
    - kafka
  environment:
    KAFKA_CLUSTERS_0_NAME: arsw-local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092

```

3.3 Levantar el entorno

```

docker compose up -d
docker ps

```

Kafka UI quedará disponible en <http://localhost:8080>. El broker Kafka quedará expuesto en localhost:9092.

3.4 Comandos básicos

```

docker exec -it arsw-kafka bash

```

```

/opt/kafka/bin/kafka-topics.sh --create --topic orders --bootstrap-server localhost:9092 --partitions 3 --
replication-factor 1

```

```

/opt/kafka/bin/kafka-topics.sh --describe --topic orders --bootstrap-server localhost:9092

```

3.5 Publicar y consumir eventos

```

/opt/kafka/bin/kafka-console-producer.sh --topic orders --bootstrap-server localhost:9092

```

```

{"orderId":"ORD-1001","customerId":"CUS-01","total":120000,"status":"CREATED"}
{"orderId":"ORD-1002","customerId":"CUS-02","total":85000,"status":"CREATED"}

```

```

/opt/kafka/bin/kafka-console-consumer.sh --topic orders --bootstrap-server localhost:9092 --from-beginning

```

3.6 Actividad práctica

Cree los topics orders, payments e inventory. Publique al menos cinco eventos JSON y verifique en Kafka UI su topic, partición, offset, clave y contenido.

Capítulo 4. Productores y consumidores con Spring Boot

En este capítulo se implementa una aplicación Spring Boot capaz de publicar y consumir eventos desde Kafka. Aunque el laboratorio usa un solo proyecto, conceptualmente cada consumidor representa un servicio lógico independiente.

4.1 Estructura del proyecto

```
src/main/java/edu/eci/arsw/kafka
├── config/KafkaTopicConfig.java
├── controller/OrderController.java
├── dto/CreateOrderRequest.java
├── dto/OrderCreatedEvent.java
├── producer/OrderEventProducer.java
└── consumer/OrderEventConsumer.java
```

4.2 Configuración application.yml

```
server:
  port: 8081

spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: order-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
    properties:
      spring.json.trusted.packages: "edu.eci.arsw.kafka.dto"
```

4.3 Evento de dominio

```
public class OrderCreatedEvent {
    private String orderId;
    private String customerId;
    private Double total;
    private String status;
    private Instant occurredAt;
    // constructores, getters y setters
}
```

4.4 Productor

```
@Service
public class OrderEventProducer {
    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

    public OrderEventProducer(KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publishOrderCreated(OrderCreatedEvent event) {
        kafkaTemplate.send("orders", event.getOrderId(), event);
    }
}
```

```

    }
}

```

4.5 Consumidor

```

@Service
public class OrderEventConsumer {
    @KafkaListener(topics = "orders", groupId = "inventory-service")
    public void consume(OrderCreatedEvent event) {
        System.out.println("Evento recibido en inventory-service: " + event.getOrderId());
    }
}

```

4.6 Endpoint REST

```

@RestController
@RequestMapping("/orders")
public class OrderController {
    private final OrderEventProducer producer;

    public OrderController(OrderEventProducer producer) {
        this.producer = producer;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public OrderCreatedEvent createOrder(@RequestBody CreateOrderRequest request) {
        OrderCreatedEvent event = new OrderCreatedEvent(
            "ORD-" + UUID.randomUUID(),
            request.getCustomerId(),
            request.getTotal(),
            "CREATED",
            Instant.now()
        );
        producer.publishOrderCreated(event);
        return event;
    }
}

```

4.7 Prueba

```

curl -X POST http://localhost:8081/orders -H "Content-Type: application/json" -d '{"customerId":"CUS-01","total":120000}'

```

Actividad 4. Trazabilidad del evento

Documente el recorrido del evento desde la solicitud HTTP hasta el consumidor. Indique topic, clave, partición, consumidor, Consumer Group y evidencia en Kafka UI.

Capítulo 5. Caso de estudio: sistema de pedidos basado en eventos

Una plataforma de comercio electrónico desea desacoplar los procesos de pedidos, pagos, inventario, facturación, notificaciones, analítica y auditoría. Kafka será utilizado como infraestructura de eventos para reducir acoplamiento y permitir procesamiento asíncrono.

5.1 Servicios propuestos

Servicio	Responsabilidad principal
order-service	Crear pedidos y publicar order-created.
payment-service	Procesar pagos y publicar payment-approved o payment-rejected.
inventory-service	Validar disponibilidad y publicar inventory-reserved o inventory-rejected.
invoice-service	Generar facturas cuando el pago sea aprobado.
notification-service	Enviar notificaciones al cliente.
analytics-service	Consumir eventos para construir indicadores.
audit-service	Registrar trazabilidad de eventos relevantes.

5.2 Topics propuestos

Topic	Eventos principales	Clave sugerida
orders	order-created, order-cancelled	orderId
payments	payment-approved, payment-rejected	orderId
inventory	inventory-reserved, inventory-rejected	orderId
invoices	invoice-generated, invoice-failed	orderId
notifications	notification-sent, notification-failed	orderId
audit	audit-record-created	correlationId

5.3 Flujo general

Cliente -> Order Service -> orders
 orders -> Payment Service -> payments
 orders -> Inventory Service -> inventory
 payments/inventory -> Notification Service
 orders/payments/inventory -> Analytics Service
 Eventos relevantes -> Audit Service

5.4 Consistencia eventual

En un sistema basado en eventos, no todos los servicios actualizan su estado de inmediato. El pedido puede iniciar en estado CREATED y avanzar hacia PAYMENT_APPROVED, INVENTORY_RESERVED, CONFIRMED o CANCELLED según los eventos recibidos.

Decisión arquitectónica

No todo proceso debe ser asíncrono. Consultar productos o autenticar usuarios puede mantenerse mediante REST, mientras que notificaciones, analítica y auditoría pueden procesarse mediante Kafka.

Actividad 5. Diseño del flujo

Proponga los eventos, topics, productores, consumidores, Consumer Groups y claves de particionamiento para el flujo de compra. Justifique por qué no conviene usar un único topic global llamado events.

Capítulo 6. Laboratorio guiado extendido

El estudiante extenderá la aplicación para que el evento order-created genere eventos posteriores de pagos e inventario. El objetivo es observar cómo un evento puede desencadenar nuevas acciones sin acoplar directamente los servicios.

6.1 Topics obligatorios

orders
payments
inventory

6.2 Eventos requeridos

Evento	Descripción
OrderCreatedEvent	Pedido creado por el order-service.
PaymentProcessedEvent	Resultado del procesamiento de pago.
InventoryProcessedEvent	Resultado de la validación de inventario.

6.3 Flujo esperado

1. Cliente crea pedido.
2. Order Service publica order-created.
3. Payment Service consume order-created y publica payment-approved o payment-rejected.
4. Inventory Service consume order-created y publica inventory-reserved o inventory-rejected.
5. Notification Service consume eventos relevantes.
6. Analytics Service registra información del flujo.

6.4 Consumidor lógico de pagos

```
@KafkaListener(topics = "orders", groupId = "payment-service")
public void consume(OrderCreatedEvent event) {
    boolean approved = event.getTotal() <= 250000;
    PaymentProcessedEvent paymentEvent = new PaymentProcessedEvent(
        "PAY-" + UUID.randomUUID(),
        event.getOrderId(),
        event.getCustomerId(),
        event.getTotal(),
        approved ? "APPROVED" : "REJECTED",
        Instant.now()
    );
    paymentProducer.publish(paymentEvent);
}
```

6.5 Consumidor lógico de inventario

```
@KafkaListener(topics = "orders", groupId = "inventory-service")
public void consume(OrderCreatedEvent event) {
    boolean reserved = event.getTotal() <= 300000;
    InventoryProcessedEvent inventoryEvent = new InventoryProcessedEvent(
        "INV-" + UUID.randomUUID(),
        event.getOrderId(),
        event.getCustomerId(),
        reserved ? "RESERVED" : "REJECTED",
        Instant.now()
    );
}
```

```
inventoryProducer.publish(inventoryEvent);
}
```

Actividad 6. Evidencia y análisis

Cree pedidos con valores diferentes y reconstruya el flujo de eventos en Kafka UI. Identifique eventos generados, topics, claves, Consumer Groups, offsets y lag.

Capítulo 7. Manejo de errores, reintentos y Dead Letter Topics

En sistemas distribuidos, los errores son inevitables. Un consumidor puede fallar por problemas temporales, errores de datos, reglas de negocio o fallos técnicos. Una solución basada en Kafka debe definir cómo actuar cuando un evento no puede procesarse correctamente.

7.1 Tipos de errores

Tipo	Ejemplo	Estrategia posible
Transitorio	Base de datos temporalmente no disponible.	Reintentar con backoff.
Permanente	Evento sin identificador obligatorio.	Enviar a DLT y revisar.
Negocio	Pago rechazado por fondos insuficientes.	Publicar evento de negocio.
Técnico	Excepción no controlada en el consumidor.	Reintentar y enviar a DLT si persiste.

7.2 Dead Letter Topic

Un Dead Letter Topic (DLT) almacena eventos que no pudieron ser procesados después de aplicar la estrategia de reintentos. No debe verse como una papelera, sino como una herramienta de diagnóstico y recuperación.

orders -> inventory-service -> falla -> reintentos -> orders.DLT

7.3 Configuración conceptual en Spring Kafka

```
DeadLetterPublishingRecoverer recoverer =
    new DeadLetterPublishingRecoverer(kafkaOperations,
        (record, exception) -> new TopicPartition(record.topic() + ".DLT", record.partition()));
```

```
FixedBackOff backOff = new FixedBackOff(2000L, 3L);
return new DefaultErrorHandler(recoverer, backOff);
```

7.4 Idempotencia

Un consumidor idempotente puede procesar el mismo evento más de una vez sin generar inconsistencias. Para lograrlo, puede utilizar eventId, restricciones únicas o una tabla de eventos procesados.

Actividad 7. Estrategia de errores

Diseñe una estrategia para manejar eventos fallidos en inventory-service. Indique cuándo reintentar, cuándo enviar a DLT, qué información revisar y cómo evitar reprocesamientos infinitos.

Capítulo 8. Buenas prácticas de diseño con Kafka

8.1 Kafka no debe usarse para todo

Kafka es útil cuando se requiere desacoplamiento, alto volumen de eventos, procesamiento asíncrono, múltiples consumidores, reprocesamiento, auditoría o analítica en tiempo real. No es necesario para consultas simples o procesos pequeños donde REST resuelve adecuadamente el problema.

8.2 Diseñar eventos como hechos

Incorrecto	Correcto
create-order	order-created
process-payment	payment-processed
reserve-inventory	inventory-reserved
send-email	notification-sent

8.3 Metadatos recomendados

```
{
  "eventId": "EVT-9001",
  "eventType": "order-created",
  "eventVersion": "1.0",
  "occurredAt": "2026-06-30T10:00:00Z",
  "source": "order-service",
  "correlationId": "CORR-1001",
  "orderId": "ORD-1001",
  "customerId": "CUS-01",
  "total": 120000
}
```

8.4 Buenas prácticas resumidas

Aspecto	Recomendación
Topics	Organizarlos por dominio y evitar un topic global sin criterio.
Claves	Elegirlas según la entidad cuyo orden importa.
Particiones	Definirlas según volumen, paralelismo y crecimiento.
Replicación	Usar factor mayor que 1 en producción.
Retención	Definirla según auditoría, reprocesamiento y almacenamiento.
Consumidores	Diseñarlos con responsabilidad clara e idempotencia.
Errores	Usar reintentos controlados y DLT.
Observabilidad	Monitorear lag, DLT, errores y tiempos de procesamiento.
Evolución	Versionar eventos y documentar contratos.

Actividad 8. Diagnóstico de buenas prácticas

Revise una arquitectura que usa un topic events, mensajes sin clave, factor de replicación 1, sin DLT y sin monitoreo de lag. Identifique problemas, atributos afectados y mejoras prioritarias.

Capítulo 9. Actividades de consolidación

Estas actividades permiten cerrar el laboratorio con decisiones arquitectónicas concretas y de alto valor.

9.1 Actividad 1. Decisiones de comunicación

Clasifique los siguientes procesos como REST, Kafka o arquitectura híbrida: consultar catálogo, crear pedido, validar pago, enviar correo, actualizar analítica, registrar auditoría, consultar estado del pedido y actualizar inventario. Justifique según respuesta inmediata, asincronía, múltiples consumidores y reprocesamiento.

9.2 Actividad 2. Diseño del flujo de eventos

Diseñe el flujo de eventos para el proceso de compra. Incluya eventos principales, productor, consumidores, topic, clave de particionamiento y Consumer Group. Responda por qué no conviene un único topic events, por qué los consumidores deben tener grupos distintos y por qué orderId puede ser una buena clave.

9.3 Actividad 3. Diagnóstico arquitectónico

Configuración propuesta:

Topic principal: events

Particiones: 1

Factor de replicación: 1

Retención: 12 horas

Mensajes sin clave

Sin eventId

Sin correlationId

Todos los consumidores en el mismo Consumer Group

Sin Dead Letter Topics

Sin monitoreo de lag

Realice un diagnóstico técnico breve: problemas identificados, atributos de calidad afectados, riesgos para producción, cambios prioritarios y propuesta de mejora.

Capítulo 10. Reto final, entregables y rúbrica

El reto final integra los conceptos trabajados durante la guía. El estudiante debe diseñar una arquitectura basada en eventos para una plataforma de comercio electrónico con pedidos, pagos, inventario, facturas, notificaciones, analítica y auditoría.

10.1 Requisitos mínimos

Elemento	Mínimo esperado
Servicios	order-service, payment-service, inventory-service, invoice-service, notification-service, analytics-service y audit-service.
Eventos	order-created, payment-approved, payment-rejected, inventory-reserved, inventory-rejected, invoice-generated, notification-sent y audit-record-created.
Diseño	Topics, productores, consumidores, Consumer Groups, claves, retención, replicación, DLT y observabilidad.

10.2 Entregable

El estudiante debe entregar un documento técnico breve con la descripción de la solución, arquitectura propuesta, tabla de servicios, tabla de eventos y topics, claves de particionamiento, Consumer Groups, estrategia de errores, riesgos y justificación de decisiones arquitectónicas.

10.3 Restricciones

- No utilizar un único topic general para todos los eventos.
- Nombrar los eventos como hechos ocurridos.
- Definir Consumer Groups coherentes por servicio.
- Incluir estrategia de errores y DLT.
- Justificar la clave de particionamiento.
- Diferenciar procesos síncronos y asíncronos.
- Incluir al menos una consideración sobre consistencia eventual.

10.4 Rúbrica

Criterio	Descripción	Peso
Diseño de eventos	Eventos bien nombrados y coherentes con el dominio.	20%
Topics y Consumer Groups	Organización adecuada y grupos definidos con criterio.	20%
Justificación arquitectónica	Relación clara con atributos de calidad.	25%
Errores y observabilidad	Incluye DLT, lag, trazabilidad y estrategia de recuperación.	15%
Claridad del entregable	Documento claro, ordenado y técnicamente comprensible.	10%
Consistencia de la propuesta	Arquitectura viable y sin contradicciones graves.	10%

10.5 Cierre de la guía

Kafka permite construir sistemas distribuidos capaces de procesar grandes volúmenes de eventos, desacoplar servicios y mejorar atributos de calidad como escalabilidad, disponibilidad, tolerancia a fallos y observabilidad. No obstante, su adopción debe responder a una decisión arquitectónica consciente. Una solución basada en eventos exige diseñar con cuidado los eventos, topics, particiones, claves, consumidores, retención, replicación, errores, trazabilidad y consistencia eventual.